COMPSCI 130 S1 2022

Introduction to Software Fundamentals

Linked Lists – The Node Class

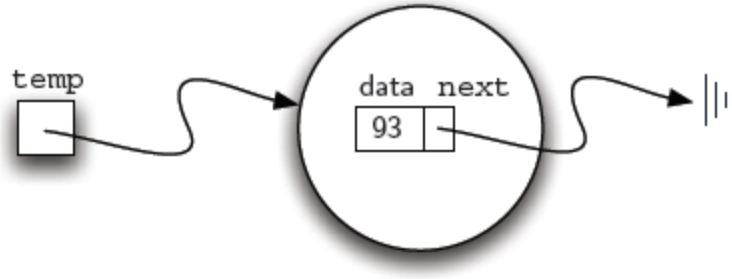
Agenda & Reading

- Agenda
  - The Node class

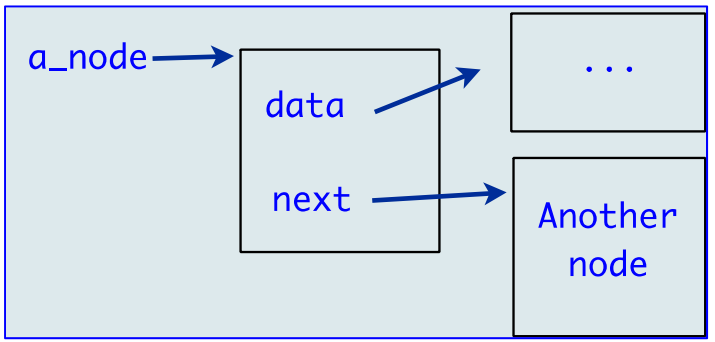
- Online Coursebook:
  - https://onlinetextbook.auckland.ac.nz/17-linked-lists/the-node-class/

The Node class

- A node
  - is the basic building block of a linked list.
  - contains the **data** as well as a **link** to the **next** node in the list.

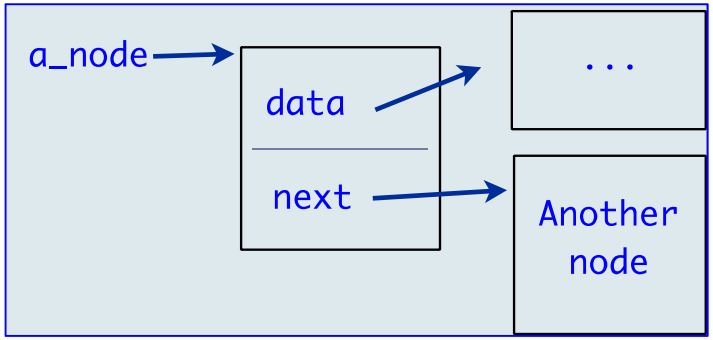



```
temp = Node(93)
```



Definition of the Node class

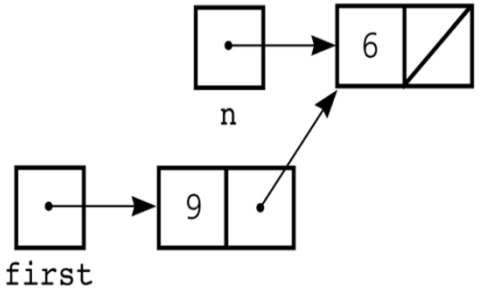
```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

    def get_data(self):
        return self.data

    def get_next(self):
        return self.next

    def set_data(self, new_data):
        self.data = new_data

    def set_next(self, new_next):
        self.next = new_next
```



2 The Node class
Chain of nodes

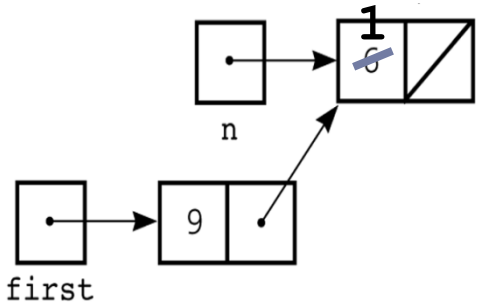
- You can build a chain of nodes using Node objects

```
n = Node(6)
first = Node(9)
first.set_next(n)
```




```
n.set_data(1)
print(first.get_next().get_data())
```

1



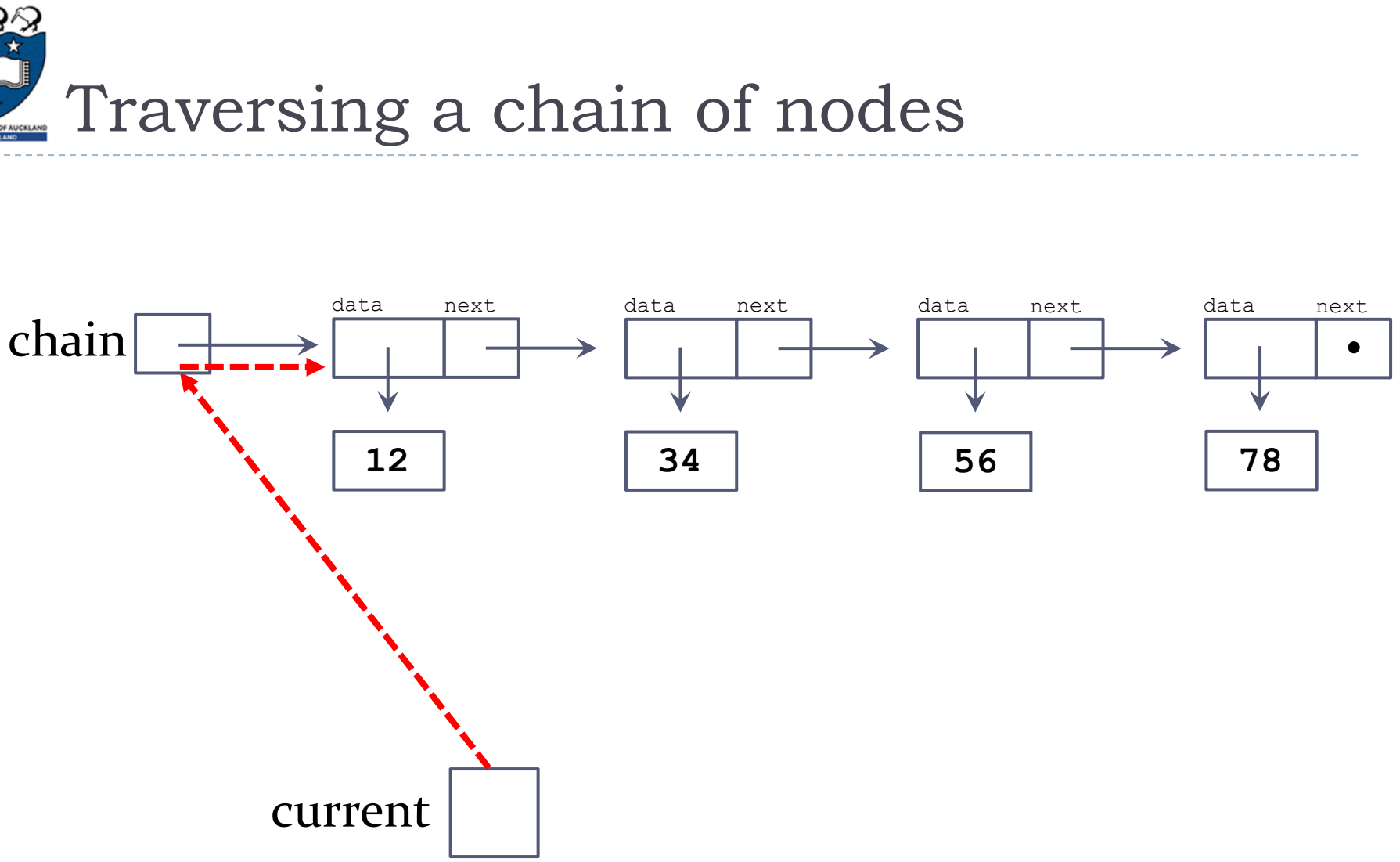
Traversing a chain of nodes

To traverse a chain of nodes:

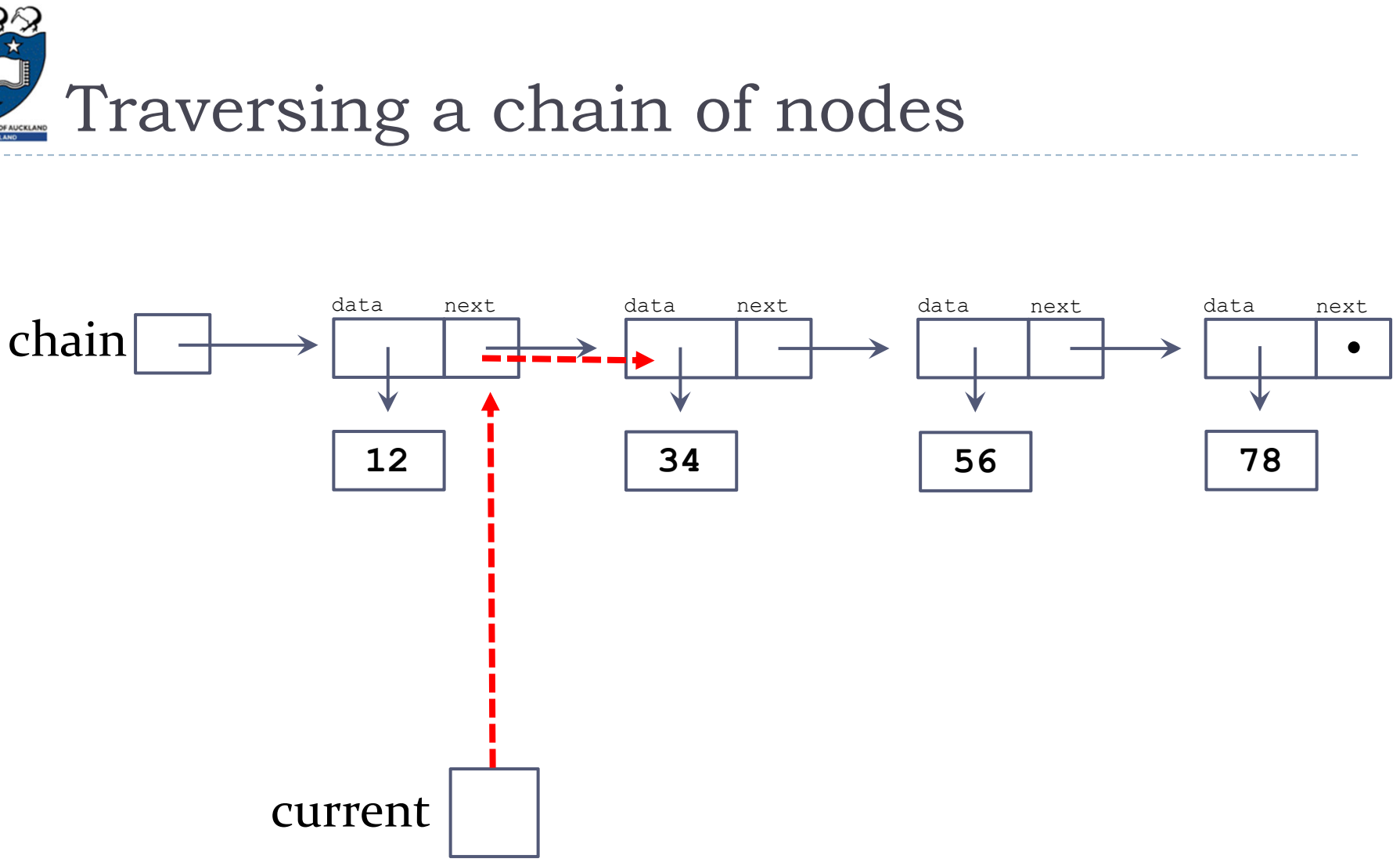
- Set a pointer to be the same address as first node in the chain.
- While the pointer is not **None:**
  - Move the pointer to the next node using the get_next() method.

Assuming that the first node in the chain has been assigned to the variable first, the following while loop will traverse the chain of nodes printing the data they store.

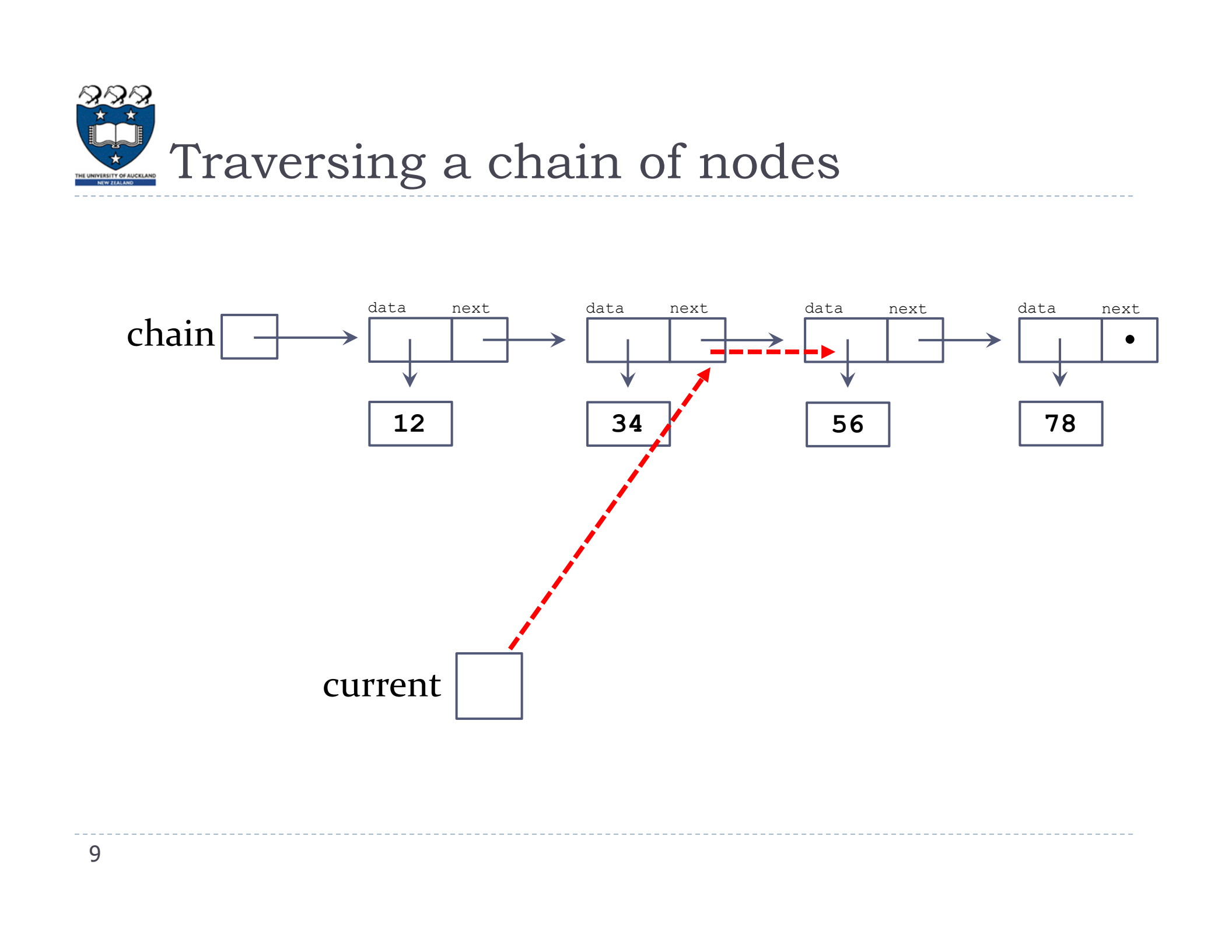
```
current = first
while current != None:
  print(current.get_data())
  current = current.get_next()
```

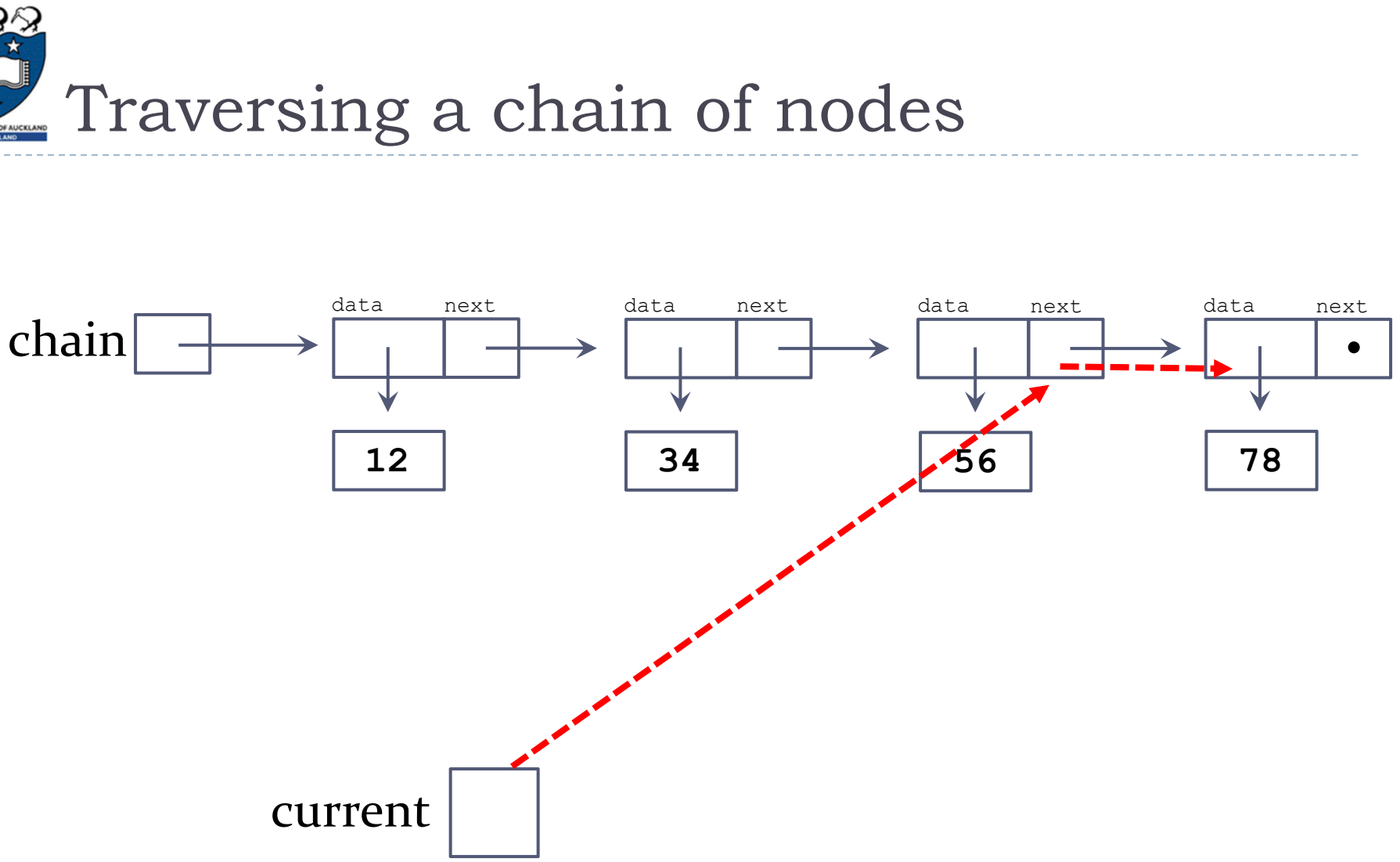


Traversing a chain of nodes



Traversing a chain of nodes


Traversing a chain of nodes
chain
data
next
data
next
data
next
data
next
12
34
56
78
current



Traversing a chain of nodes

Exercise 1

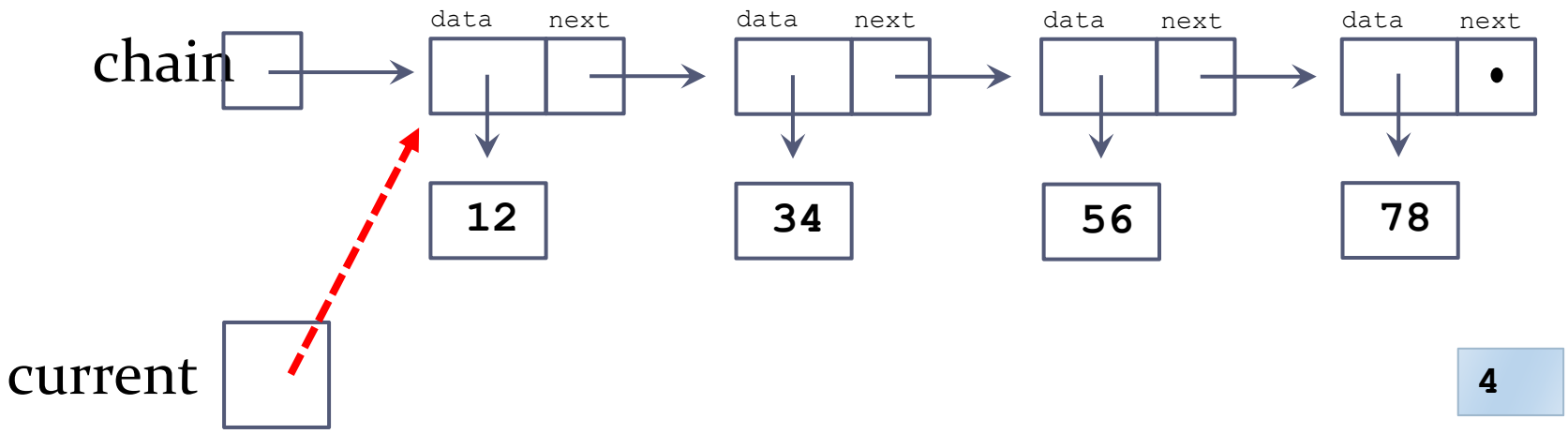
- What is the output of the following program?

```
def print_chain(n):
    while not n == None:
        print(n.get_data(), end = " ")
        n = n.get_next()
    print()

n5 = Node(15)
n6 = Node(34)
n7 = Node(12)
n8 = Node(84)

n6.set_next(n5)
n7.set_next(n8)
n8.set_next(n6)
n5.set_next(None)

print_chain(n5)
print_chain(n8)
```

```
A) 12 84
B) 12 84 34 15
C) 12 15 34 84
D) 84 34 15 None
E) 84 34 15 12
```

Exercise 2

- Define a function named **`get_count(node)`** which takes a **`Node`** object as a parameter and returns the number of nodes in this chain of nodes



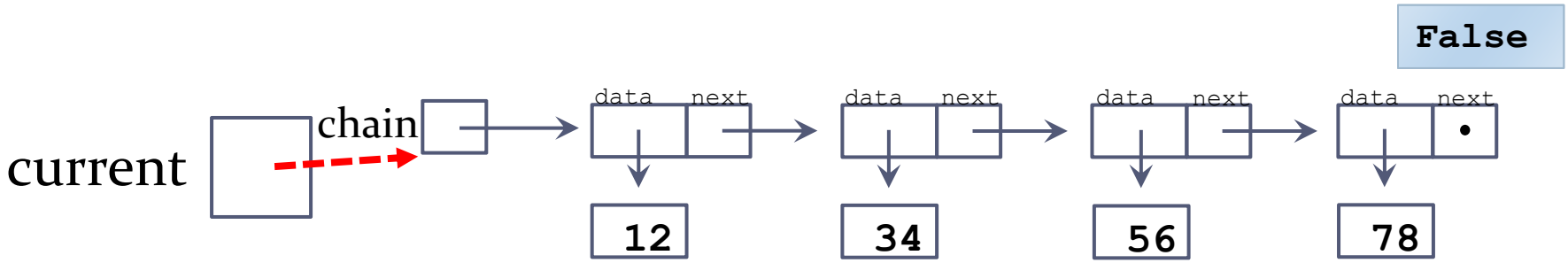
4

```
def get_count(node):
    count = 0
    current = node
    while




    return count
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
    def get_data(self):
        return self.data
    def get_next(self):
        return self.next
```

Exercise 3

- Define a function named no_evens(node) which takes a Node object as a parameter and returns True if all values are odd numbers in this chain of nodes, and returns False otherwise
  - If encounter an even number, then stop and return False
  - Otherwise, continue to the end and return True




```
def no_evens(node):
    current = node
    while current:
        if


    return True
```

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
    def get_data(self):
        return self.data
    def get_next(self):
        return self.next
```